HDOC Day-72

Topic: Student Attendance Management Using a Dynamically Resizable Matrix in C

Objective: Create a pointer-based attendance matrix using malloc(), add students using realloc(), validate attendance values, calculate percentages and free allocated memory.

Complete C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int readInt(const char *message, int min, int max) {
    int n, ch;
    while (1) {
        printf("%s", message);
        if (scanf("%d", &n) == 1 && n >= min && n <= max)
            return n;
        printf("Invalid input. Try again.\n");
        while ((ch = getchar()) != '\n' && ch != EOF) { }
        if (ch == EOF) exit(EXIT_FAILURE);
    }
}

int **createMatrix(int students, int days) {
    int **a = malloc((size_t)students * sizeof *a);
    if (a == NULL) return NULL;
    for (int i = 0; i < students; i++) {
        a[i] = malloc((size_t)days * sizeof *a[i]);
        if (a[i] == NULL) {
            for (int j = 0; j < i; j++) free(a[j]);
            free(a);
            return NULL;
        }
    }
    return a;
}

void recordAttendance(int **a, int start, int end, int days) {
    for (int i = start; i < end; i++) {
        printf("\nStudent %d:\n", i + 1);
        for (int j = 0; j < days; j++) {
            char prompt[80];
            snprintf(prompt, sizeof prompt,
                     "  Day %d (1=Present, 0=Absent): ", j + 1);
            *(*(a + i) + j) = readInt(prompt, 0, 1);
        }
    }
}

int addStudents(int ***matrix, int *students, int extra, int days) {
    int old = *students;
    if (extra > INT_MAX - old) return 0;
    int total = old + extra;
    int **temp = realloc(*matrix, (size_t)total * sizeof **matrix);
    if (temp == NULL) return 0;
    *matrix = temp;

    for (int i = old; i < total; i++) (*matrix)[i] = NULL;
    for (int i = old; i < total; i++) {
        (*matrix)[i] = malloc((size_t)days * sizeof ***matrix);
        if ((*matrix)[i] == NULL) {
            for (int j = old; j < i; j++) {
                free((*matrix)[j]);
                (*matrix)[j] = NULL;
            }
            return 0;
        }
    }
    *students = total;
```

```
    recordAttendance(*matrix, old, total, days);
    return 1;
}

double percentage(int *row, int days) {
    int present = 0;
    for (int j = 0; j < days; j++) present += *(row + j);
    return 100.0 * present / days;
}

void displayAttendance(int **a, int students, int days) {
    printf("\nATTENDANCE RECORD\n");
    for (int i = 0; i < students; i++) {
        printf("Student %d: ", i + 1);
        for (int j = 0; j < days; j++)
            printf("%d ", *(*(a + i) + j));
        printf(" | %.2f%%\n", percentage(*(a + i), days));
    }
}

void releaseMemory(int **a, int students) {
    for (int i = 0; i < students; i++) free(*(a + i));
    free(a);
}

int main(void) {
    int students = readInt("Number of students: ", 1, 10000);
    int days = readInt("Number of working days: ", 1, 10000);
    int **attendance = createMatrix(students, days);
    if (attendance == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }
    recordAttendance(attendance, 0, students, days);
    while (1) {
        int choice = readInt("\n1. Add students\n2. Display records"
                             "\n3. Exit\nChoice: ", 1, 3);

        if (choice == 3) break;
        if (choice == 2) {
            displayAttendance(attendance, students, days);
        } else {
            int extra = readInt("How many students to add? ", 1, 10000);
            if (!addStudents(&attendance, &students, extra, days))
                printf("Could not add students (memory/size limit).\n");
        }
    }
    releaseMemory(attendance, students);
    return 0;
}
```

Sample Run

```
Number of students: 2
Number of working days: 3
Student 1: 1 0 1
Student 2: 1 1 1
Choice: 1
How many students to add? 1
Student 3: 0 1 1
Choice: 2
ATTENDANCE RECORD
Student 1: 1 0 1  | 66.67%
Student 2: 1 1 1  | 100.00%
Student 3: 0 1 1  | 66.67%
Choice: 3
```

Result: The program successfully stores attendance in dynamically allocated rows, expands the matrix when students are added, displays each student's attendance percentage and releases memory before exiting.